TVET Online Pastpapers Database - Frontend API Guide

This document is written for the frontend developer who will connect a plain JavaScript frontend to the Pastpapers backend.

The backend is built with plain PHP and returns JSON. The frontend should use `fetch()` from a normal HTML page that loads a JavaScript file.

1. Start Here

Use this as the basic page structure:

```html
<!DOCTYPE html>
<html>
    <head>
        <title>TVET Online Pastpaers database</title>
        <script type="module" src="index.js"></script>
    </head>
    <body>
        <div id="main"></div>
    </body>
</html>
```

What this file does

- Declares a normal HTML page.
- Loads `index.js` as a module.
- Gives JavaScript one main container: `#main`.
- Keeps the frontend simple and framework-free.

Suggested file structure for the frontend developer

```text
public_html/
├── index.html
├── index.js
├── styles.css
├── assets/
│   ├── logo.png
│   └── icons/
└── pages/
    ├── login.js
    ├── register.js
    ├── dashboard.js
    └── questions.js
```

You can keep everything inside one `index.js` if you want, but separating features into smaller files is easier to maintain.

2. Backend Base URL

The backend endpoints are exposed from the root of the domain.

If your website is:

```text
https://pastpapers.kwartisans.com
```

Then the API URLs are:

- `https://pastpapers.kwartisans.com/register`
- `https://pastpapers.kwartisans.com/login`
- `https://pastpapers.kwartisans.com/logout`
- `https://pastpapers.kwartisans.com/questions`
- `https://pastpapers.kwartisans.com/answers`
- `https://pastpapers.kwartisans.com/vote`
- `https://pastpapers.kwartisans.com/leaderboard`
- `https://pastpapers.kwartisans.com/dashboard`
- `https://pastpapers.kwartisans.com/admin/users`
- `https://pastpapers.kwartisans.com/admin/stats`

Do not call `register.php`, `login.php`, or other `.php` files from the frontend. Use the clean URLs above.

Recommended base constant

In `index.js`:

```javascript
const API_BASE = 'https://pastpapers.kwartisans.com';
```

If you later move the project to another domain, you only need to change this one line.

3. How the Frontend Should Work

The frontend should follow this flow:

1. Show a login or register screen.
2. Send form data to the backend with `fetch()`.
3. Save the logged-in user in memory or local storage if needed.
4. Load questions after login.
5. Allow users to open a question.
6. Allow users to answer the question.
7. Allow users to vote on answers.
8. Show dashboard stats and leaderboard.

The backend returns JSON only. The frontend should read the JSON, check `success`, and update the UI.

4. General `fetch()` Rules

Use these rules for every request:

- Send JSON bodies with `Content-Type: application/json`.
- Use `POST` for register, login, answers, votes, and mark-correct.
- Use `GET` for lists and dashboard data.
- Handle `401` and `403` responses.
- Handle `404` when an endpoint or record is missing.
- Show readable error messages to the user.

Common helper function

```javascript
async function request(url, options = {}) {
    const response = await fetch(url, {
        headers: {
            'Content-Type': 'application/json',
            ...(options.headers || {})
        },
        ...options
    });

    const data = await response.json();

    if (!response.ok) {
        throw new Error(data.message || 'Request failed');
    }

    return data;
}
```

This helper keeps the frontend code clean.

5. Authentication Endpoints

5.1 Register

- **URL:** `/register`
- **Method:** `POST`
- **Purpose:** Create a new user account.

Request body

```json
{
  "name": "Jane Student",
  "email": "jane@school.com",
  "password": "user123"
}
```

JavaScript example

```javascript
async function registerUser(name, email, password) {
    const response = await fetch(`${API_BASE}/register`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ name, email, password })
    });

    const data = await response.json();
    return data;
}
```

Success response

```json
{
  "success": true,
  "message": "Registration successful",
  "user": {
    "id": "...",
    "email": "jane@school.com",
    "name": "Jane Student",
    "role": "user"
  }
}
```

Failure response

```json
{
  "success": false,
  "message": "Email already registered"
}
```

Frontend notes

- Validate empty fields before sending.
- Show a loading state while waiting.
- Redirect the user to login or dashboard after success.

5.2 Login

- **URL:** `/login`
- **Method:** `POST`
- **Purpose:** Authenticate a user and start a session.

Request body

```json
{
  "email": "jane@school.com",
  "password": "user123"
}
```

JavaScript example

```javascript
async function loginUser(email, password) {
    const response = await fetch(`${API_BASE}/login`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ email, password })
    });

    const data = await response.json();

    if (data.success) {
        localStorage.setItem('currentUser', JSON.stringify(data.user));
    }

    return data;
}
```

Success response

```json
{
  "success": true,
  "message": "Login successful",
  "user": {
    "id": "...",
    "name": "Jane Student",
    "email": "jane@school.com",
    "role": "user"
  }
}
```

Failure response

```json
{
  "success": false,
  "message": "Invalid credentials"
}
```

Frontend notes

- If login fails, show the backend message.
- Do not store the password in local storage.
- The backend session cookie should stay in the browser automatically.

5.3 Logout

- **URL:** `/logout`
- **Method:** `POST`
- **Purpose:** End the current session.

JavaScript example

```javascript
async function logoutUser() {
    const response = await fetch(`${API_BASE}/logout`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        }
    });

    const data = await response.json();
    localStorage.removeItem('currentUser');
    return data;
}
```

6. Questions Endpoints

6.1 Get All Questions

- **URL:** `/questions`
- **Method:** `GET`
- **Purpose:** Return all available questions.

Example usage

```javascript
async function getQuestions() {
    const response = await fetch(`${API_BASE}/questions`);
    return await response.json();
}
```

Success response shape

```json
{
  "success": true,
  "data": [
    {
      "id": "...",
      "title": "Database Fundamentals",
      "description": "...",
      "questions": [],
      "difficulty": "medium",
      "answers_count": 3,
      "top_answer": null
    }
  ]
}
```

Frontend notes

- Render the list as cards or rows.
- Show title, difficulty, and answer count.
- Add a button to open the question detail view.

6.2 Get Question Detail

- **URL:** `/questions/{id}`
- **Method:** `GET`
- **Purpose:** Load one question and its answers.

Example usage

```javascript
async function getQuestionDetail(questionId) {
    const response = await fetch(`${API_BASE}/questions/${questionId}`);
    return await response.json();
}
```

Success response shape

```json
{
  "success": true,
  "data": {
    "id": "...",
    "title": "Database Fundamentals",
    "description": "...",
    "questions": [],
    "answers_count": 2,
    "answers": [
      {
        "id": "...",
        "content": "...",
        "votes": 5,
        "is_correct": false,
        "user_name": "John Doe"
      }
    ]
  }
}
```

Frontend notes

- Display the question title and description first.
- List answers sorted by votes.
- Highlight the answer marked as correct.

6.3 Post a New Question

- **URL:** `/questions`
- **Method:** `POST`
- **Purpose:** Admin posts a new pastpaper question set.
- **Access:** Admin only.

Request body

```json
{
  "title": "TVET Networking Past Paper",
  "description": "Module 1 exam questions",
  "difficulty": "medium",
  "questions": [
    {
      "question": "What is TCP?",
      "options": ["Protocol", "Device", "Cable", "Software"],
      "correct_answer": "Protocol"
    }
  ]
}
```

JavaScript example

```javascript
async function createQuestion(payload) {
    const response = await fetch(`${API_BASE}/questions`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(payload)
    });

    return await response.json();
}
```

Frontend notes

- Validate that `questions` is an array.
- Let admin paste or build a JSON array.
- Show validation errors before sending.

7. Answers Endpoints

7.1 Get Answers for a Question

- **URL:** `/answers?question_id={id}`
- **Method:** `GET`
- **Purpose:** Load answers for one question.

Example usage

```javascript
async function getAnswers(questionId) {
    const response = await fetch(`${API_BASE}/answers?question_id=${encodeURIComponen
    return await response.json();
}
```

7.2 Post a New Answer

- **URL:** `/answers`
- **Method:** `POST`
- **Purpose:** Logged-in users submit answers.

Request body

```json
{
  "question_id": "123",
  "content": "TCP is a communication protocol used on networks."
}
```

JavaScript example

```javascript
async function postAnswer(questionId, content) {
    const response = await fetch(`${API_BASE}/answers`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ question_id: questionId, content })
    });

    return await response.json();
}
```

Success response

```json
{
  "success": true,
  "message": "Answer posted successfully"
}
```

Frontend notes

- Disable the submit button while posting.
- Clear the form after success.
- Refresh the answers list after posting.

7.3 Mark Answer as Correct

- **URL:** `/mark-correct`
- **Method:** `POST`
- **Purpose:** Question owner marks the best answer as correct.

Request body

```json
{
  "question_id": "123",
  "answer_id": "456"
}
```

JavaScript example

```javascript
async function markCorrect(questionId, answerId) {
    const response = await fetch(`${API_BASE}/mark-correct`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ question_id: questionId, answer_id: answerId })
    });

    return await response.json();
}
```

Frontend notes

- Show this action only to the question author or admin.
- Refresh the question detail after success.

8. Voting Endpoint

Vote on an Answer

- **URL:** `/vote`

- **Method:** `POST`
- **Purpose:** Upvote an answer.

Request body

```json
{
  "answer_id": "456"
}
```

JavaScript example

```javascript
async function voteAnswer(answerId) {
    const response = await fetch(`${API_BASE}/vote`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ answer_id: answerId })
    });

    return await response.json();
}
```

Frontend notes

- Prevent multiple clicks while the request is in progress.
- Show the new vote count after success.
- If the backend says the user already voted, show that message clearly.

9. Dashboard Endpoint

User Dashboard

- **URL:** `/dashboard`
- **Method:** `GET`
- **Purpose:** Return the logged-in user's activity and reputation.

Example usage

```javascript
async function loadDashboard() {
    const response = await fetch(`${API_BASE}/dashboard`);
    return await response.json();
}
```

Expected response shape

```json
{
  "success": true,
  "data": {
    "user": {
      "id": "...",
      "name": "Jane Student",
      "email": "jane@school.com",
      "role": "user"
    },
    "reputation": {
      "total_points": 35,
      "answers_count": 3,
      "correct_answers": 1,
      "total_votes": 12
    },
    "activity": {
      "answered_questions": 3,
      "posted_questions": 1,
      "badges": []
    },
    "recent_answers": [],
    "recent_questions": []
  }
}
```

Frontend notes

- Show reputation as a large number.
- Show answer count, correct count, and total votes in cards.
- Show recent activity in a table or timeline.

10. Leaderboard Endpoint

Get Leaderboard

- **URL:** `/leaderboard`
- **Method:** `GET`
- **Purpose:** Show top contributors.

Optional query parameters

- `limit` - number of results to show.
- `page` - pagination page.

Example:

```text
/leaderboard?limit=10&page=1
```

Example usage

```javascript
async function loadLeaderboard() {
    const response = await fetch(`${API_BASE}/leaderboard?limit=10&page=1`);
    return await response.json();
}
```

Frontend notes

- Sort display by rank.
- Show avatar, name, and reputation.
- Add paging controls if you want a long list.

11. Admin Endpoints

11.1 Admin Users

- **URL:** `/admin/users`
- **Method:** `GET` or `PUT`
- **Purpose:** List or update users.
- **Access:** Admin only.

List users example

```javascript
async function loadUsers() {
    const response = await fetch(`${API_BASE}/admin/users`);
    return await response.json();
}
```

Update user example

```javascript
async function updateUser(userId, role, status) {
    const response = await fetch(`${API_BASE}/admin/users`, {
        method: 'PUT',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ user_id: userId, role, status })
    });

    return await response.json();
}
```

11.2 Admin Stats

- **URL:** `/admin/stats`
- **Method:** `GET`
- **Purpose:** Show platform analytics.
- **Access:** Admin only.

Example usage

```javascript
async function loadStats() {
    const response = await fetch(`${API_BASE}/admin/stats`);
    return await response.json();
}
```

Frontend notes

- Show total users, questions, answers, and votes.
- Show questions by difficulty.
- Show top contributors.

12. Full Plain JavaScript Setup Example

This is a simple example of how the frontend developer can wire everything together.

```javascript
```

```
const API_BASE = 'https://pastpapers.kwartisans.com';
const main = document.getElementById('main');

document.addEventListener('DOMContentLoaded', () => {
    renderLoginPage();
});

function renderLoginPage() {
    main.innerHTML = `
        <section>
            <h1>Login</h1>
            <form id="loginForm">
                <input type="email" id="email" placeholder="Email" required />
                <input type="password" id="password" placeholder="Password" required
                <button type="submit">Login</button>
            </form>
            <p id="message"></p>
        </section>
    `;

    document.getElementById('loginForm').addEventListener('submit', handleLogin);
}

async function handleLogin(event) {
    event.preventDefault();

    const email = document.getElementById('email').value;
    const password = document.getElementById('password').value;
    const message = document.getElementById('message');

    try {
        const response = await fetch(`${API_BASE}/login`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify({ email, password })
        });

        const data = await response.json();

        if (!data.success) {
            message.textContent = data.message;
```

```
            return;
        }

        localStorage.setItem('currentUser', JSON.stringify(data.user));
        renderDashboard();
    } catch (error) {
        message.textContent = error.message;
    }
}

async function renderDashboard() {
    const response = await fetch(`${API_BASE}/dashboard`);
    const data = await response.json();

    if (!data.success) {
        main.innerHTML = `<p>${data.message}</p>`;
        return;
    }

    main.innerHTML = `
        <section>
            <h1>Dashboard</h1>
            <p>Reputation: ${data.data.reputation.total_points}</p>
            <p>Answers: ${data.data.reputation.answers_count}</p>
            <p>Correct Answers: ${data.data.reputation.correct_answers}</p>
        </section>
    `;
}
```

13. Recommended UI Sections

The frontend should ideally have these sections:

- Login page
- Register page
- Questions list page
- Question detail page
- Submit answer form
- Dashboard page
- Leaderboard page
- Admin dashboard page

You can use one-page navigation or separate screens. Both are fine with plain JavaScript.

14. Error Handling Strategy

The backend returns messages like:

- `Invalid credentials`
- `Missing required fields`
- `Unauthorized`
- `Forbidden`
- `Question not found`

The frontend should:

1. Read the JSON response.
2. Check `success`.
3. Display the backend message if `success` is false.
4. Avoid generic alerts when a message is already available.

Error helper

```javascript
function showMessage(element, message, type = 'info') {
    element.textContent = message;
    element.className = type;
}
```

15. Session and Login State

This backend uses PHP sessions.

That means:

- The browser keeps the session cookie automatically.
- The frontend does not need to manually send tokens.
- After login, requests to `/dashboard`, `/answers`, `/vote`, and admin routes will work because the session is already stored in the browser.

Important note

If your frontend is hosted on a different domain, you may need extra CORS and cookie settings. If frontend and backend are on the same domain, this is simpler.

16. Data Shapes the Frontend Should Expect

Question object

```json
{
  "id": "string",
  "title": "string",
  "description": "string",
  "questions": [],
  "difficulty": "easy|medium|hard",
  "answers_count": 0,
  "top_answer": null
}
```

Answer object

```json
{
  "id": "string",
  "question_id": "string",
  "content": "string",
  "answered_by": "string",
  "votes": 0,
  "is_correct": false,
  "user_name": "string"
}
```

User object

```json
{
  "id": "string",
  "name": "string",
  "email": "string",
  "role": "user|admin"
}
```

17. Frontend Developer Checklist

Before shipping the frontend, make sure of the following:

- `index.html` loads `index.js` correctly.
- `API_BASE` points to the live domain.

- Login and register forms send JSON.
- Questions list loads from `/questions`.
- Question detail loads from `/questions/{id}`.
- Answer posting calls `/answers`.
- Voting calls `/vote`.
- Dashboard calls `/dashboard`.
- Admin pages call `/admin/users` and `/admin/stats`.
- Error messages are visible to the user.
- Loading states are shown during network requests.
- The UI still works on mobile screens.

18. Common Mistakes

Mistake 1: Calling the old `.php` files

Wrong:

```javascript
fetch('/backend/api/login.php')
```

Correct:

```javascript
fetch('/login')
```

Mistake 2: Sending form data instead of JSON

Wrong:

```javascript
body: new FormData(form)
```

Correct:

```javascript
body: JSON.stringify({ email, password })
```

Mistake 3: Forgetting the JSON header

```javascript
headers: {
    'Content-Type': 'application/json'
}
```

Mistake 4: Not checking `success`

Always do this:

```javascript
const data = await response.json();

if (!data.success) {
    alert(data.message);
    return;
}
```

19. Minimal End-to-End Example

This is the shortest flow for a frontend developer.

1. User opens `index.html`.
2. `index.js` loads and renders the login form.
3. User submits email and password.
4. Frontend POSTs to `/login`.
5. Backend returns JSON.
6. If successful, frontend stores the current user and loads `/questions`.
7. User opens one question.
8. Frontend loads `/questions/{id}` and `/answers?question_id={id}`.
9. User submits an answer with `/answers`.
10. Another user votes with `/vote`.
11. The question owner marks the best answer with `/mark-correct`.
12. Dashboard and leaderboard update from the same session.

20. Final Notes

This backend is designed for plain JavaScript, plain HTML, and PHP hosting.

If you keep the frontend simple and consistent with the API contract, integration should be straightforward.

The most important rules are:

- Use the clean endpoints.
- Send JSON.
- Check `success`.
- Handle errors properly.
- Keep the UI modular.

If you need to extend this later, the safest approach is to keep the same response shape and add new fields rather than changing existing ones.